

Universidade Federal do Ceará - Campus Quixadá
QXD0010 – Estruturas de Dados
Prof. Atílio G. Luiz

AVALIAÇÃO CONTINUADA 03

A solução da atividade descrita neste documento deve ser entregue até as 23h59 do dia **02/11/2025** via Moodle. **Cada aluno deve apresentar os resultados obtidos para o professor em data marcada posteriormente.**

Leia atentamente as instruções abaixo.

Instruções:

- Esta atividade **DEVE** ser feita **INDIVIDUALMENTE** e implementada usando a linguagem de programação C.
- O seu trabalho deve ser compactado (**.gz**, **.tar**, **.zip**, **.rar**) e enviado pelo Moodle.
- Identifique o seu código-fonte colocando o seu **nome** e **matrícula** como comentário no topo de cada arquivo do seu projeto.
- Indente corretamente o seu código para facilitar o entendimento.
- O código-fonte deve estar devidamente **organizado** e **documentado**.
- Esta avaliação vale de **0 a 30 pontos**.
- **Observação:** Se você alocar memória dinamicamente, lembre-se de desalocar os endereços de memória alocados quando os mesmos não forem mais ser usados.
- **Observação:** Qualquer indício de plágio resultará em nota **ZERO** para todos os envolvidos.

DICA: COMECE O TRABALHO O QUANTO ANTES.

1 Problema: Comparando empiricamente o tempo de execução de algoritmos de ordenação

Neste trabalho, deve-se implementar os seguintes algoritmos de ordenação:

- BubbleSort, InsertionSort, SelectionSort, MergeSort, QuickSort e mais um sexto algoritmo de sua escolha (com exceção do CockTailSort).

Você deve programar em C:

- (1) Uma versão para cada um dos cinco algoritmos acima usando **vetor de inteiros**.

Além disso, você deve:

- (2) Pesquisar um outro algoritmo de ordenação de seu interesse, diferente dos já apresentados, e implementá-lo.

Obs.: devem ser implementados e executados o total de **6 algoritmos** de ordenação.

A atividade é individual.

2 Testes

Você deve comparar diferentes estratégias de ordenação para ordenar um conjunto de N inteiros positivos, **aleatoriamente gerados**. Realize experimentos considerando listas de inteiros aleatoriamente geradas com tamanho $N = 1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000, 11000, 12000, 13000, 14000, 15000, 16000, 17000, 18000, 19000, 20000, 21000, 22000, 23000, 24000, 25000, 26000, 27000, 28000, 29000, 30000, 31000, 32000, 33000, 34000, 35000, 36000, 37000, 38000, 39000, 40000, 41000, 42000, 43000, 44000, 45000, 46000, 47000, 48000, 49000, 50000, 51000, 52000, 53000, 54000, 55000, 56000, 57000, 58000, 59000, 60000, 61000, 62000, 63000, 64000, 65000, 66000, 67000, 68000, 69000, 70000, 71000, 72000, 73000, 74000, 75000, 76000, 77000, 78000, 79000, 80000, 81000, 82000, 83000, 84000, 85000, 86000, 87000, 88000, 89000, 90000, 91000, 92000, 93000, 94000, 95000, 96000, 97000, 98000$ e 99000 . Para cada valor de N , realize experimentos com 5 **sementes** diferentes. Para a comparação dos algoritmos de ordenação, avalie:

- os **valores médios do tempo de execução**¹.

No relatório, você deve apresentar uma pequena comparação entre os algoritmos/implementações. Apresente gráficos com os resultados obtidos. Discuta os resultados e conclusões obtidas. No seu experimento, qual algoritmo teve melhor desempenho? Você sabe dizer por quê? Você sabe interpretar os resultados obtidos?

¹Existem algumas bibliotecas em C que facilitam gerar números aleatórios em C, por exemplo, pesquise por **stdlib.h** e **time.h**.

Observação: Juntamente com esta descrição do trabalho, foi disponibilizado no Moodle um pequeno programa-exemplo, com a implementação usual do algoritmo BubbleSort e de um outro algoritmo que ordena vetores de inteiros chamado **CocktailSort**. No programa-exemplo, 495 vetores de inteiros gerados aleatoriamente são ordenados usando os algoritmos BubbleSort e CocktailSort (são 495 porque são gerados 5 vetores de tamanho N para cada um dos 99 possíveis valores de N listados acima; logo, $495 = 99 \cdot 5$). No programa fornecido são calculados os valores médios dos tempos de execução. Para cada valor de N estipulado acima, são gerados 5 vetores aleatórios de tamanho N e a média do tempo das cinco execuções do algoritmo sobre esses cinco vetores de tamanho N é armazenada em um arquivo.

Para cada execução do CocktailSort e do BubbleSort, é calculada a média do seu tempo de execução em microssegundos e esses dados são gravados em arquivos chamados **resultadoCocktail.txt** e **resultadoBubble.txt** (que encontram-se na pasta **resultados**). Cada um desses arquivos é composto de duas colunas: a primeira indica o tamanho do vetor e a segunda indica o tempo médio em microssegundos que o respectivo algoritmo levou para ordenar o respectivo vetor.

2.1 Gerando os gráficos

Uma vez gerado o arquivo **resultadoCocktail.txt**, por exemplo, você pode usar este arquivo para gerar o gráfico usando o gerador de gráficos (plotador) que você quiser. Se você nunca fez isso na vida e não conhece nenhum gerador de gráficos, há alguns escritos em Python e que são razoavelmente fáceis de usar. Procure por **Matplotlib** na internet.

Pois bem, com o arquivo **resultadoCocktail.txt** em mãos, é possível usar um plotador para plotar(desenhar) um gráfico que mostre a relação entre o tamanho do vetor gerado e o tempo em microssegundos que levou para o CocktailSort ordenar o vetor. Eu usei o **Matplotlib** do Python para gerar os seguintes gráficos² das Figuras 1 e 2.

²Os arquivos usados para gerar os gráficos no python estão na pasta **graficos**. Os arquivos que o programa python lê são os arquivos com extensão **.py** que estão na pasta **graficos**, ou seja, os arquivos **graphBubble.py**, **graphCocktail.py** e **graphBubbleCocktail.py**; esses arquivos leem os dados que estão nos arquivos da pasta **resultados** e então geram os gráficos.

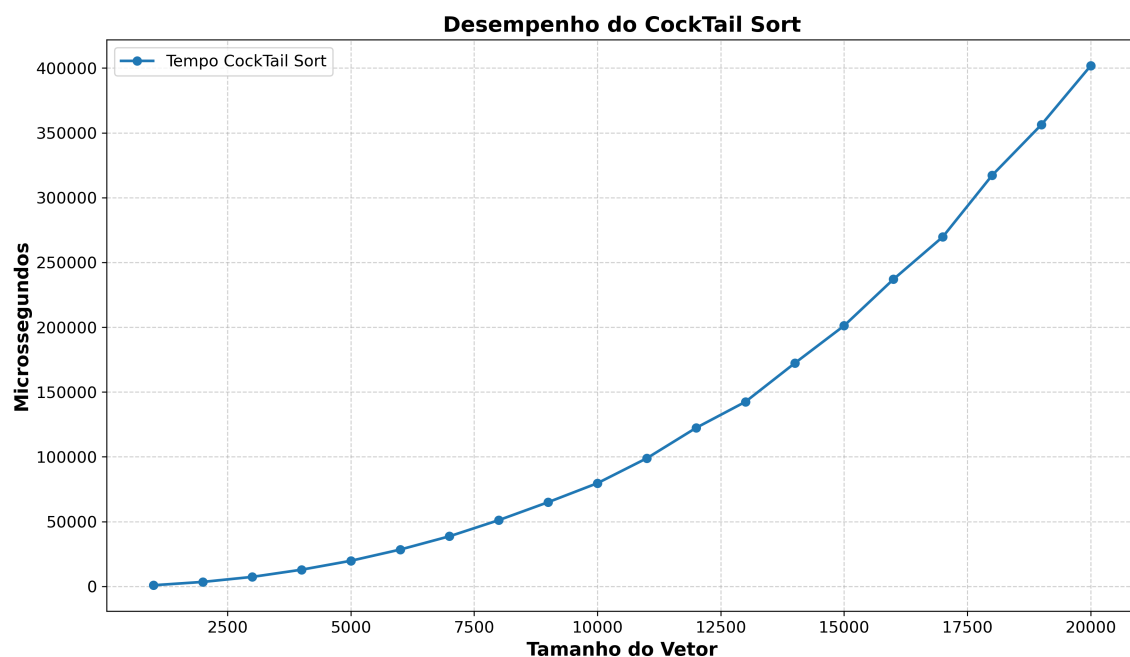


Figura 1: CocktailSort

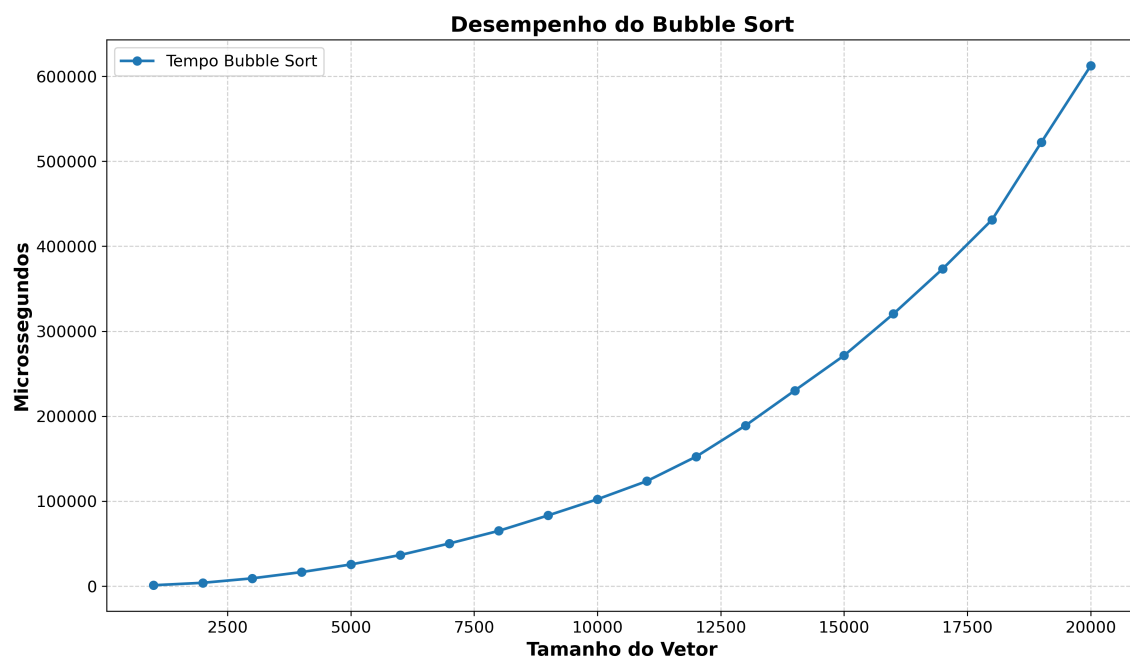


Figura 2: BubbleSort

No gráfico da Figura 3, os tempos de execução do BubbleSort e do CockTail Sort são plotados no mesmo gráfico a fim de compararmos visualmente as suas performances.

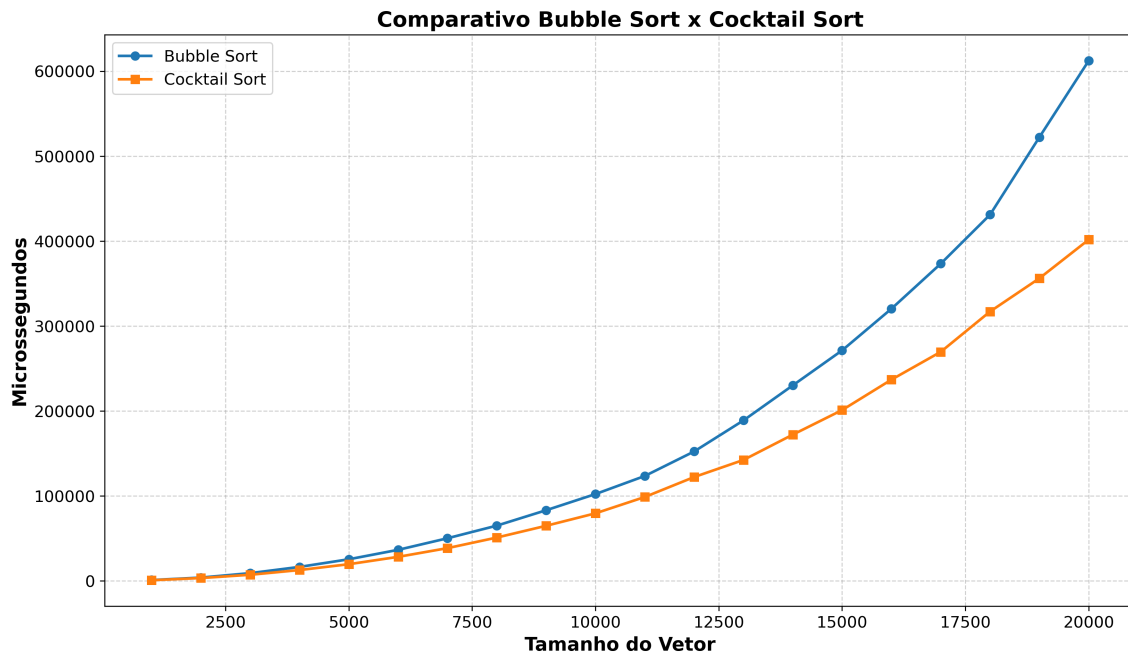


Figura 3: BubbleSort × CockTailSort. O CockTailSort acaba sendo mais rápido que o BubbleSort.

3 Informações adicionais

- Deverá ser submetido, juntamente com o código, um **relatório técnico** contendo:
 - (1) Uma descrição breve das principais características de cada um dos 6 algoritmos de ordenação que foram programados, como por exemplo:
 - (i) qual a complexidade de pior caso?
 - (ii) são estáveis?
 - (iii) são algoritmos *in loco*?
 - (iv) são recursivos ou iterativos?
 - (v) Qual a ideia principal do algoritmo? Como ele faz para ordenar o vetor?
 - (2) Gráficos mostrando os tempos de execução dos algoritmos para distintos tamanhos de entrada; e sua interpretação dos dados que são mostrados;
 - (3) Uma explicação do algoritmo que você pesquisou e implementou.
 - (4) Comparação gráfica entre os tempos de execução dos 6 algoritmos. Se não couber os 6 numa mesma figura, você pode agrupá-los em duplas de figuras. **Atenção:** cada gráfico deve vir acompanhado de um texto seu interpretando os resultados e discursando os motivos de tal gráfico dar tal resultado.
 - (5) Uma seção de dificuldades encontradas.
 - (6) Uma seção de bibliografia contendo as referências utilizadas. **Se você consultar algum site da internet, vídeo ou livro, coloque esta fonte de pesquisa no seu trabalho. Não há porque omitir as consultas.**
- O relatório deve ser entregue em formato PDF.

- A apresentação presencial da atividade e do relatório será feita em horário definido pelo professor.